3. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process\n");
        printf("PID  : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        sleep(10);
    }
    else
    {
        printf("Parent Process\n");
        printf("PID  : %d\n", getpid());

        wait(NULL);
    }

    return 0;
}
```

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i;

    for(i=0;i<3;i++)
    {
        if(fork()==0)
        {
            printf("Child %d PID=%d\n",i+1,getpid());
            return 0;
        }
    }

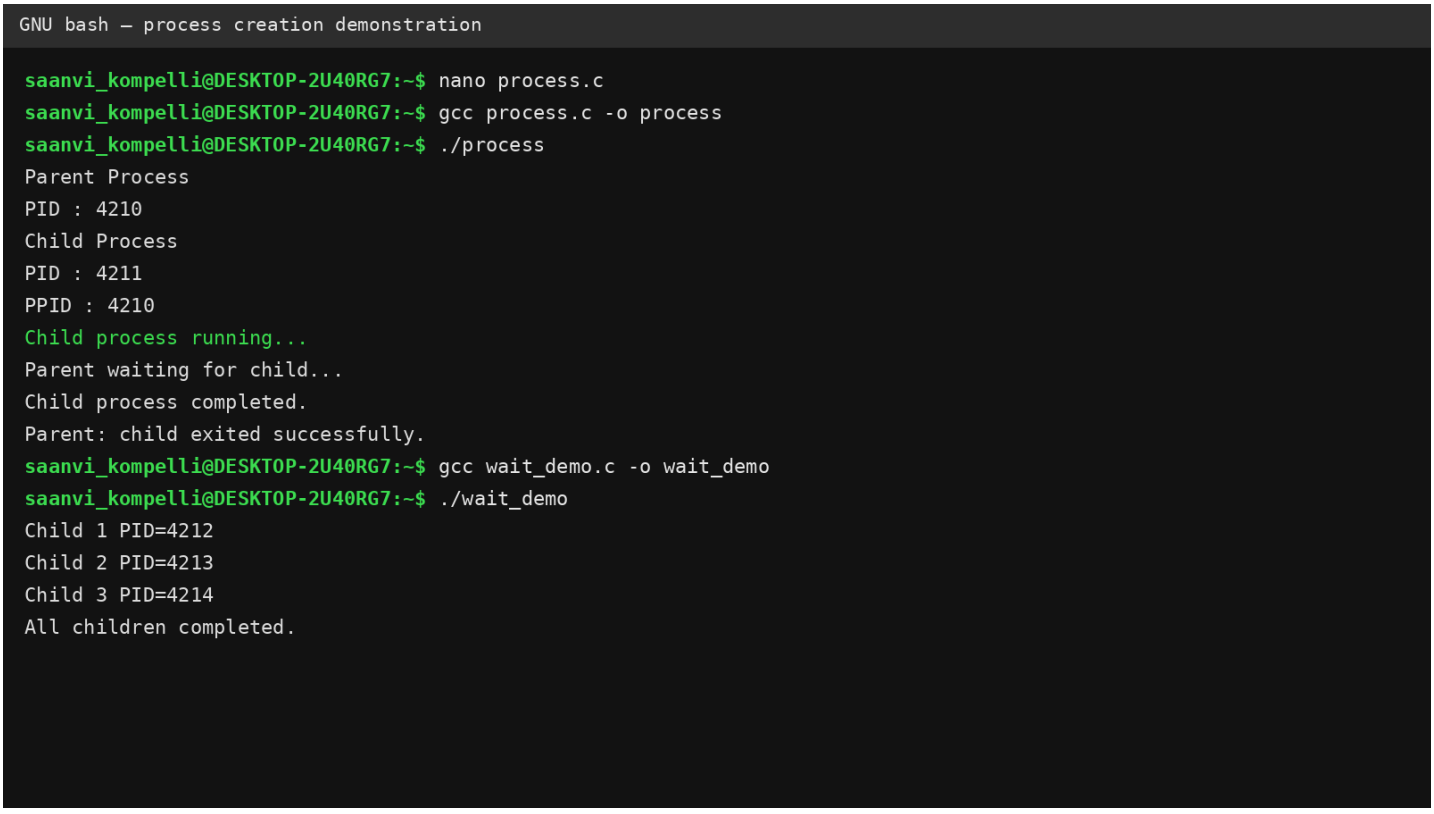
    for(i=0;i<3;i++)
    {
        wait(NULL);
    }

    printf("All children completed.\n");

    return 0;
}
```

Week 3 & 4 — Execution Example 1

Illustrative terminal output with the requested username (generated example).



Week 3 & 4 — Execution Example 2

Illustrative terminal output with the requested username (generated example).

